

MODULE 3 · WEEK 2

Web Scraping & ETL

Go get the data that doesn't exist yet: scrape it, extract fields with RegEx, and load a clean DataFrame for modeling.

Lectures 25–26 · 28 · 34 · 36 · 39



This week

- 1 **ETL** — Extract, Transform, Load
- 2 **Scraping** — pulling data off the web with requests + BeautifulSoup
- 3 **RegEx** — extracting price, year, make/model, mileage
- 4 **Cleaning** — turning messy strings into typed columns
- 5 **Champions** — train models on the scraped dataset

What is ETL?

- **Extract** raw data from a source, **Transform** (clean/standardize) it, **Load** it somewhere useful.
- Not every dataset is a clean CSV — often you have to build it yourself.
- If you can scrape a site and turn it into a clean DataFrame, you can get data on almost anything.

Key idea ETL is most of the real job: the model is easy once the data is clean.

Scraping the web

- **requests** fetches the page; **BeautifulSoup** parses the HTML.
- Locate the elements that hold your data and pull out their text.
- Be a good citizen — **rate-limit** your requests so you don't hammer the server.

```
import requests; from bs4 import BeautifulSoup
html = requests.get(url).text
soup = BeautifulSoup(html, 'html.parser')
listings = soup.select('.listing')
```

Extracting fields with RegEx

- Scraped text is messy — "\$14,995", "2018", "72k miles" all live in one blob.
- **Regular expressions** match patterns and pull out exactly the piece you want.
- Build one pattern per field: PRICE_RE, YEAR_RE, MAKE_MODEL_RE, mileage.

```
import re
PRICE_RE = re.compile(r'\$([\d,]+)')
YEAR_RE  = re.compile(r'\b(19|20)\d{2}\b')
price = int(PRICE_RE.search(text).group(1).replace(',', '', ''))
```

Key idea One tight RegEx per field turns a text blob into clean, typed columns.



Three flavors of mileage (clean carefully)

- The same value appears as "72,000", "72k", or "72000 mi" — your parser must handle all three.
- Standardize to one numeric format before modeling.
- Edge cases are where ETL quietly breaks — test your extractor on real, ugly rows.



From clean data to champion models

- Once the scraped data is a clean DataFrame, it's a normal ML problem again.
- Train and compare models (the "champions") to predict car price.
- Save the cleaned data to CSV so downstream steps don't re-scrape.

Lab this week

- Use requests + BeautifulSoup to scrape car listings, clean them with pandas + regex, and save a CSV.
- Then train and compare champion models on the dataset you built.

In the labs repo: `Module3/Week2_Scraping_ETL/scrape_cars.py` · `Module3/Week2_Scraping_ETL/train_champions.py` · `Module3/Week2_Scraping_ETL/CarPricePipeline_ChampionModels.ipynb`



Key takeaways

- ETL = Extract, Transform, Load — building the dataset is most of the work.
- requests fetches, BeautifulSoup parses, and rate-limiting keeps you a good citizen.
- RegEx extracts each field (price, year, make/model, mileage) from messy text.
- Standardize edge cases before modeling, then train champions on your clean data.